bp-enrisher

Enrichissement des Business Partners français avec les identifiants fiscaux manquants (`SIREN` , `SIRET` , `TVA`) à partir des exports SAP, des API publiques françaises et d'un matching fuzzy sur le nom et l'adresse.

Objectif

Le script identifie les BP français dont la fiche SAP est incomplète sur les identifiants fiscaux, tente de reconstituer les valeurs manquantes, puis génère :

- un fichier complet de travail ;
- un fichier réduit contenant uniquement les BP pour lesquels une correction a été trouvée ;
- un email de synthèse avec ou sans pièce jointe selon le résultat.

Le flux principal est implémenté dans `main.py` .

Ce que fait le projet

Le programme suit cette séquence :

1. Charger les exports SAP `BUT000` , `BUT020` , `ADRC` et `TAXNUM` .
2. Normaliser les identifiants BP / adresse / SIREN / SIRET.
3. Reconstituer la table BP + adresse + identifiants fiscaux.
4. Filtrer le périmètre utile : BP FR.
5. Déterminer quelles valeurs sont manquantes.
6. Tenter un enrichissement par plusieurs stratégies.
7. Exporter le résultat complet et les seules lignes modifiées.
8. Envoyer un email de notification.

Structure du dossier

Fichier	Rôle
`main.py`	Script principal : chargement, préparation, enrichissement, exports, email
`enrish_bp.py`	Logique d'enrichissement ligne par ligne
`get_best_match.py`	Sélection du meilleur candidat via scoring fuzzy nom/adresse
`request_by.py`	Appels vers l'API `recherche-entreprises` par code postal, département, nom ou coordonnées
`recherche_soc_api.py`	Wrapper HTTP générique avec retry/backoff
`recherche_cord.py`	Géocodage d'adresse via `api-adresse.data.gouv.fr`
`requestFromSiren.py`	Recherche du meilleur SIRET à partir d'un SIREN
`format.py`	Fonctions de nettoyage des noms, codes postaux et composants d'adresse
`DATA_DICTIONARY.md`	Dictionnaire de données synthétique
`requirements.txt`	Dépendances Python

Prérequis

Le script suppose :

- un accès aux exports SAP sur le partage réseau ;
- un accès réseau sortant vers les API publiques françaises ;
- la disponibilité dans l'environnement Python des modules internes `logger` et `mails` importés par `main.py` ;

Dépendances

`requirements.txt` contient :

- `pandas`
- `requests`
- `thefuzz`
- `openpyxl`

- `-e \\snt-python\Apps\Lib`

Installation

Exemple d'installation locale :

```
cd bp-enrisher
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Exécution

```
cd bp-enrisher
py -3 main.py
```

Sources de données

Les chemins sont définis en dur dans `main.py` :

- `BUT_000` : `\\interfacessap.file.core.windows.net\interfacess4p\data_mdm_export\BP_BUT000.csv`
- `BUT_020` : `\\interfacessap.file.core.windows.net\interfacess4p\data_mdm_export\BP_BUT020.csv`
- `ADRC` : `\\interfacessap.file.core.windows.net\interfacess4p\data_mdm_export\BP_ADRC.csv`
- `CREDS` : `\\interfacessap.file.core.windows.net\interfacess4p\data_mdm_export\BP_TAXNUM.csv`
- `OUTPUT_DIR` : `\\snetor-docs\Users\MDM\998_CHecks\BP-ENRISHED_BP`

Colonnes exploitées

`BP_BUT000.csv`

Colonnes chargées explicitement :

Colonne	Usage
`Business Partner`	Identifiant BP
`Grp.`	Groupe BP utilisé pour le filtrage
`Search Term 1`	Filtre d'exclusion sur les entrées `#`
`Name 1`	Nom principal utilisé pour le matching
`Last Name` / `First Name`	Reconstitution des personnes physiques si `Name 1` est vide
`Date`	Métadonnée BP
`User`	Métadonnée BP

`BP_BUT020.csv`

Colonne	Usage
`Business Partner`	Clé de jointure BP
`Addr. No.`	Clé de jointure vers ADRC

`BP_ADRC.csv`

Colonnes renommées après lecture :

Colonne finale	Usage
`street`	Adresse principale
`street4`	Complément adresse
`street5`	Complément adresse
`city`	Ville
`postcode`	Code postal
`country`	Pays

`BP_TAXNUM.csv`

Le fichier est réduit à 3 colonnes :

Colonne	Usage
`BP`	Clé BP
`value`	Valeur de l'identifiant
`type`	Type SAP de l'identifiant

Le pivot produit ensuite :

Type SAP	Colonne finale
`FR0`	`VAT`
`FR1`	`siret`
`FR2`	`siren`

Normalisation des données

Avant toute jointure, le script :

- convertit les identifiants en texte ;
- supprime les suffixes `.0` issus d'exports Excel/CSV ;
- retire les caractères non numériques pour les identifiants ;
- padde les BP à 10 chiffres quand possible ;
- garde les adresses sous une forme exploitable pour le géocodage et le matching.

Cas particulier des personnes physiques

Si `Name 1` est vide mais que `Last Name` et/ou `First Name` sont renseignés, le script reconstruit :

```
Name 1 = Last Name + " " + First Name
```

Les enregistrements commençant par `#` restent exclus du périmètre.

Construction du dataset

La fonction `build_data()` réalise les étapes suivantes :

1. Lecture de `BUT000`.
2. Lecture de `BUT020`.
3. Lecture de `ADRC`.
4. Jointure BP -> adresse via `merge_address()`.
5. Lecture de `BP_TAXNUM.csv`.
6. Filtrage des types `FR0`, `FR1`, `FR2`.
7. Pivot des identifiants fiscaux.
8. Fusion finale BP + adresse + identifiants.
9. Détection des identifiants manquants.

Schéma logique :

```
BUT000 --[BP]--> BUT020 --[Addr. No.]--> ADRC
   |
   +--[BP]--> TAXNUM pivoté (VAT / siret / siren)
```

Règles de filtrage

Le périmètre final conserve uniquement les BP jugés pertinents pour l'enrichissement :

- groupe commençant par `ZG` ;
- groupes numériques `01` à `13` ;
- exclusion du groupe `ZG11` ;
- pays `FR` quand la colonne `country` est présente ;
- exclusion des noms commençant par `#` ;
- exclusion des BP commençant par `0005`.

Détection des valeurs manquantes

Le script crée trois colonnes booléennes :

- `missing siren`
- `missing siret`
- `missing vat`

Une valeur est considérée manquante si elle est :

- vide ;
- `None` ;
- `nan` ;
- `n/a` ;
- non numérique quand un identifiant numérique est attendu.

Logique d'enrichissement

L'enrichissement s'effectue ligne par ligne dans `enrish_bp.py`.

Cas 1 : `siren`, `siret` et `vat` manquants

Le script lance une recherche externe complète via `find_siren()` selon cet ordre :

1. recherche par `nom + code postal` ;
2. recherche par `nom + département` ;
3. recherche par `nom seul` ;
4. géocodage de l'adresse puis recherche `near_point` avec rayons successifs `0.01`, `0.05`, `0.1`.

Si un candidat crédible est trouvé :

- `matching siren` est renseigné ;
- `matching siret` est renseigné ;
- `missing siren` passe à `False` ;
- `missing siret` passe à `False`.

La TVA est ensuite calculée si elle est toujours manquante.

Cas 2 : `siren` manquant mais `siret` présent

Le SIREN est déduit directement des 9 premiers caractères du SIRET.

Cas 3 : `siren` manquant mais `VAT` présente

Le SIREN est déduit depuis la TVA française :

```
siren = VAT[4:]
```

Cas 4 : `vat` manquante mais `siren` ou `siret` disponible

La TVA française est recalculée avec :

```
clé = (12 + 3 * (SIREN % 97)) % 97
TVA = "FR" + clé + SIREN
```

Cas 5 : `siret` manquant mais `siren` disponible

Le script appelle l'API SIRENE et choisit l'établissement dont l'adresse ressemble le plus à l'adresse SAP.

Matching fuzzy

`get_best_match.py` compare les candidats retournés par l'API avec la ligne SAP à partir :

- du nom ;
- de l'adresse ;
- de la ville.

Pondérations utilisées :

Signal	Poids
Nom	`0.5`
Rue / adresse	`0.35`
Ville	`0.15`

Colonnes de score produites :

Colonne	Signification
`score`	Score combiné final
`name score`	Similitude sur le nom
`street score`	Similitude sur l'adresse
`supposed right`	`True` si `name score >= TRESHOLD (85)`

Géocodage

Quand les recherches par nom échouent, `recherche_cord.py` tente de géocoder la ligne avec :

- `street`
- `city`
- `postcode`

Le module nettoie certains marqueurs parasites :

- `CEDEX`
- `CS`
- `BP`

Si le géocodage avec code postal échoue, le script retente sans code postal.

APIs utilisées

API	URL	Usage
Recherche entreprises	`https://recherche-entreprises.api.gouv.fr/search`	Recherche par nom / code postal / département
Recherche par coordonnées	`https://recherche-entreprises.api.gouv.fr/near_point`	Recherche d'entreprises autour d'un point géocodé
API Adresse	`https://api-adresse.data.gouv.fr/search`	Géocodage de l'adresse SAP
API SIRENE	`https://api-avis-situation-sirene.insee.fr/identification/siren/{siren}`	Recherche des établissements d'un SIREN

Politique de retry

`recherche_soc_api.py` applique :

- jusqu'à `3` tentatives ;
- un backoff de `5` secondes ;
- un timeout HTTP de `15` secondes par appel.

`requestFromSiren.py` applique également une boucle de retry plus longue côté SIRENE.

Sorties générées

Chaque exécution crée un dossier horodaté sous :

```
\\snetor-docs\Users\MDM\998_CHecks\BP-ENRISHED_BP\YYYY-MM-DD_HH-MM_search\
```

Fichiers produits :

Fichier	Contenu
`frenchBPs.xlsx`	Dataset de travail après filtrage, avant enrichissement
`missing_siren_found.xlsx`	Résultat complet après enrichissement
`missing_siren_found_to_edit.xlsx`	Uniquement les lignes dont au moins un statut `missing` a changé

Schéma de sortie principal

Le fichier `missing_siren_found.xlsx` est réordonné avec les colonnes suivantes :

Colonne	Description
`BP`	Identifiant Business Partner
`Name 1`	Nom BP utilisé pour le matching
`original missing siren`	Statut initial
`original missing siret`	Statut initial
`original missing vat`	Statut initial

Colonne	Description
`missing siren`	Statut final après enrichissement
`missing siret`	Statut final après enrichissement
`missing vat`	Statut final après enrichissement
`matching siren`	SIREN trouvé ou déduit
`matching siret`	SIRET trouvé ou déduit
`matching vat`	TVA calculée ou trouvée
`siren`	SIREN source
`siret`	SIRET source
`VAT`	TVA source
`score`	Score global du meilleur match
`name score`	Score de similarité nom
`street score`	Score de similarité adresse
`supposed right`	Indicateur de confiance simple

Email envoyé

Sujet :

```
BP Enrichment
```

Comportement :

- si aucune ligne n'a été enrichie : email sans pièce jointe avec le message `Aucun BP n'a ete corrige.`
- si au moins une ligne a été enrichie : email avec `missing_siren_found_to_edit.xlsx`

Logs

Le traitement s'appuie sur le module interne `logger` et journalise :

- les chargements de fichiers ;
- les volumes de données ;
- les stratégies de recherche tentées ;
- les réponses des API ;
- les erreurs et exceptions.

Points de configuration

Modifier les chemins d'entrée / sortie

Dans `main.py` :

- `BUT_000`
- `BUT_020`
- `ADRC`
- `CREDS`
- `OUTPUT_DIR`

Modifier les endpoints

Endpoint	Fichier	Variable / fonction
Recherche par nom	`recherche_soc_api.py`	`API_URL`
Recherche par coordonnées	`request_by.py`	`request_by_coordinates()`
Géocodage adresse	`recherche_cord.py`	`url`
API SIRENE	`requestFromSiren.py`	construction de `url`

Email

Le script principal appelle `send_quality_check_mail`. La configuration dépend du module interne `mails` et, selon l'environnement, d'un `config.cfg` ou d'identifiants Graph disponibles localement.

Limites connues

- le script ne modifie pas SAP ; il produit uniquement une proposition de correction ;
- la qualité du résultat dépend fortement de la qualité du nom et de l'adresse source ;
- le matching fuzzy peut générer des faux positifs si plusieurs sociétés proches existent ;
- `supposed right` reste un indicateur simple, pas une validation métier ;
- plusieurs composants (`logger` , `mails`) ne sont pas embarqués dans ce dossier et doivent exister dans le runtime.

Référence complémentaire

Voir aussi `DATA_DICTIONARY.md` pour un résumé orienté colonnes.